Final Project - Overview and Review Criteria

Estimated time needed: 5 minutes

Project Overview

For this final project, you will assume the role of a software engineer who needs to develop an AI-based web application. You will analyze the scenario and perform the following tasks:

- Task 1: Clone the project repository
- Task 2: Create an emotion detection application using Watson NLP library
- Task 3: Format the output of the application
- Task 4: Package the application
- Task 5: Run Unit tests on your application
- Task 6: Web deployment of the application using Flask
- Task 7: Incorporate error handling
- Task 8: Run static code analysis

Evaluation Criteria – 16 Points

You can submit your project deliverables in one of the following ways:

- **Option 1: AI-Graded Submission and Evaluation**
  When you choose Option 1, you will be redirected to an AI tool where you can upload your deliverables, which may include URLs, terminal outputs, code snippets, or screenshots. You will then receive an AI-generated grade that will automatically reflect on your progress page.

- **Option 2: Peer-Graded Submission and Evaluation**
  When you choose Option 2, you will upload your deliverables—such as URLs, terminal outputs, code snippets, or screenshots—through the My Submission section. Your submission will then be reviewed either by your peers or by AI grader.

  We recommend using Option 1 for faster grading. However, if you face any issues or cannot access it, you may use Option 2 instead.

  If you encounter any grading problems, please reach out to the Course Team through the Discussion Forums.

Please find the deatils of the Grading Criteria below:

▼ Criteria for Option 1: AI-Graded Submission and Evaluation:

Task 1: Submit the GitHub repository URL

- Submit the public GitHub repository URL of the README.md file, which contains the project name details. (1 point)

Task 2: Create an emotion detection application using the Watson NLP library

- Activity 1: Submit the code from the emotion_detection.py file that shows the application function. (1 point)
- Activity 2: Submit the terminal output showing that the application was imported and tested without errors. (1 point)

Task 3: Format the output of the application

- Activity 1: Submit the code from the emotion_detection.py file showing that the modified emotion_detector function returns the correct output format. (1 point)
- Activity 2: Submit the terminal output showing that the output format is accurate. (1 point)

Task 4: Validate the EmotionDetection package

- Activity 1: Submit the public GitHub repository URL of the **init**.py file which includes the code to import the application module. (1 point)
- Activity 2: Submit the terminal output validating that EmotionDetection is a valid package. (1 point)

Task 5: Run unit tests on your application

- Activity 1: Submit the code from the test_emotion_detection.py file showing the required unit tests. (1 point)
- Activity 2: Submit the terminal output showing that all unit tests passed. (1 point)

Task 6: Web deployment of the application using Flask

- Activity 1: Submit the code from the server.py file showing the web deployment of the application using Flask. (1 point)
- Activity 2: Upload the screenshot named 6b_deployment_test.png showing the application deployment. (1 point)

Task 7: Incorporate error handling

- Activity 1: Submit the code from the emotion_detection.py file showing the updated emotion_detector function for status code 400. (1 point)
- Activity 2: Submit the code from the server.py file showing the handling of blank input errors. (1 point)
- Activity 3: Upload the screenshot named 7c_error_handling_interface.png validating error handling functionality. (1 point)

Task 8: Run static code analysis

- Activity 1: Submit the code from the server.py file demonstrating the execution of static code analysis. (1 point)
- Activity 2: Submit the terminal output showing a perfect score for static code analysis. (1 point)

▼ Criteria for Option 2: Peer-Graded Submission and Evaluation

You will need to provide clear screenshots for each of the activities mentioned below for grading.

Task1: Clone the project repository

- Repository folder structure (1 point)

Task 2: Create an emotion detection application using Watson NLP library

- Activity 1: Write code for the application function (1 point)
- Activity 2: Import the application without errors (1 point)

Task 3: Format the output of the application

- Activity 1: Modify the emotion_predictor function accurately for it to return the provided output format (1 point)
- Activity 2: Check that the output format is accurate (1 point)

Task 4: Package the application

- Activity 1: Create the folder structure for packaging the application and create the **init**.py file (1 point)
- Activity 2: Verify that the output is as expected (1 point)

Task 5: Run Unit tests on your application

- Activity 1: Create the required unit tests (1 point)
- Activity 2: Run user tests until all pass (1 point)

Task 6: Web deployment of the application using Flask

- Activity 1: Update server.py with the right contents (1 point)
- Activity 2: Deploy the application (1 point)

Task 7: Incorporate Error handling

- Activity 1: Update the emotion_detector function for status code, 400 (1 point)
- Activity 2: Update the server.py file to manage blank input errors (1 point)
- Activity 3: Validate error handling functionality (1 point)

Task 8: Run static code analysis

- Activity 1: Update the server.py to highlight the code that gets 10/10 (1 point)
- Activity 2: Get a perfect score for static code analysis (1 point)

Author(s)

Ratima Raj Singh

Skills Network